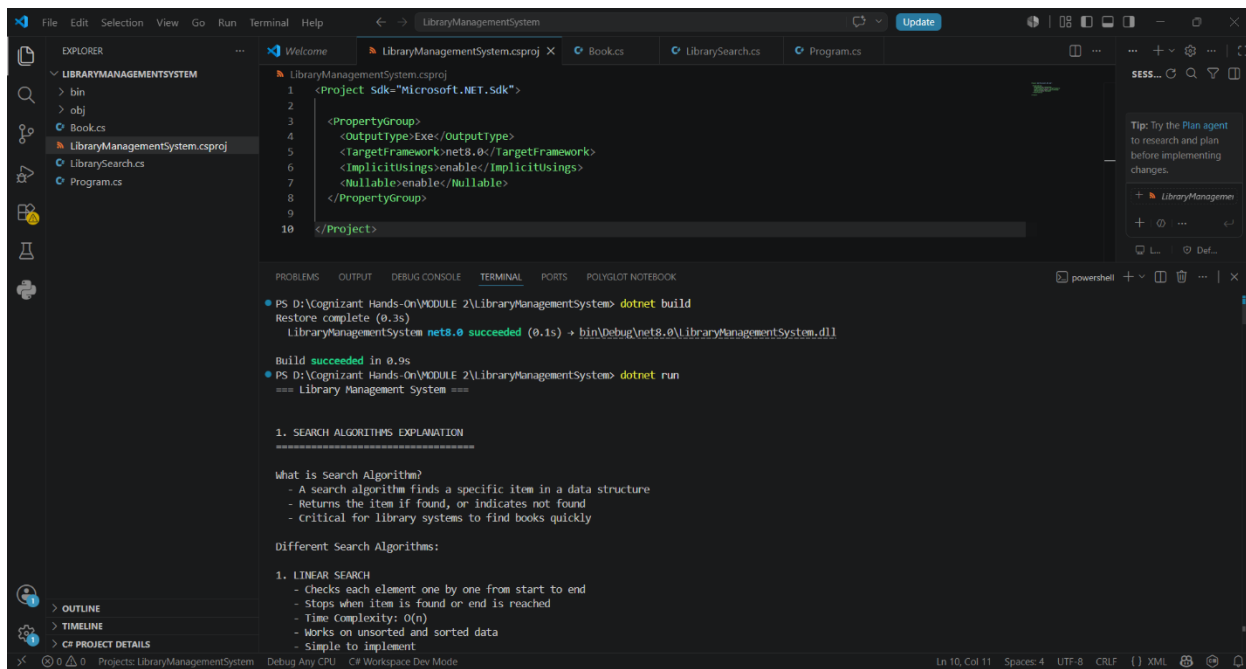


Hands-On

Exercise 6: Library Management System

In this module, I was able to successfully implement, execute and test a Library Management System in .NET has developed a project called LibraryManagementSystem. To manage book information effectively, I developed a Book class that has attributes like BookId, Title, and Author. Learned about search algorithms like Linear Search and Binary Search, their working and their applications. I then added two search methods, Linear Search and Binary Search, to search for books by title in the system. Once implemented, I performed various search operations and tested them to get the correct book records. I also analyzed the time complexity of both search methods and knew that Binary Search is more efficient for sorted data sets, and Linear Search is more efficient on smaller or unsorted data sets. This hands-on experience gave me a good insight into the significance of search algorithms and choosing the right ones for creating efficient and maintainable .NET applications.

Coding:



```
File Edit Selection View Go Run Terminal Help
LibraryManagementSystem
Update
EXPLORER
LIBRARYMANAGEMENTSYSTEM
  bin
  obj
  Book.cs
  LibraryManagementSystem.csproj
  LibrarySearch.cs
  Program.cs
SOLUTION EXPLORER
LibraryManagementSystem.csproj
  <Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
TERMINAL
PS D:\Cognizant Hands-On\MODULE 2\LibraryManagementSystem> dotnet build
Restore complete (0.3s)
LibraryManagementSystem net8.0 succeeded (0.1s) -> bin\Debug\net8.0\LibraryManagementSystem.dll
Build succeeded in 0.9s
PS D:\Cognizant Hands-On\MODULE 2\LibraryManagementSystem> dotnet run
== Library Management System ==

1. SEARCH ALGORITHMS EXPLANATION

What is Search Algorithm?
- A search algorithm finds a specific item in a data structure
- Returns the item if found, or indicates not found
- critical for library systems to find books quickly

Different Search Algorithms:

1. LINEAR SEARCH
- Checks each element one by one from start to end
- Stops when item is found or end is reached
- Time Complexity: O(n)
- Works on unsorted and sorted data
- Simple to implement
```

```
1 using System;
2 using System.Collections.Generic;
3
4 namespace LibraryManagementSystem
5 {
6     // 20 references
7     public class LibrarySearch
8     {
9         // ===== LINEAR SEARCH =====
10        // 2 references
11        public static Book? LinearSearchByTitle(Book[] books, string title)
12        {
13            for (int i = 0; i < books.Length; i++)
14            {
15                if (books[i].Title == title)
16                {
17                    return books[i];
18                }
19            }
20            return null;
21        }
22
23        // 1 reference
24        public static List<Book> LinearSearchByTitlePartial(Book[] books, string title)
25        {
26            List<Book> results = new List<Book>();
27            for (int i = 0; i < books.Length; i++)
28            {
29                if (books[i].Title.ToLower().Contains(title.ToLower()))
30                {
31                    results.Add(books[i]);
32                }
33            }
34            return results;
35        }
36    }
37
38    // 1 reference
39    public static Book? LinearSearchByAuthor(Book[] books, string author)
40    {
41        // ...
42    }
43 }
```

```
1 namespace LibraryManagementSystem
2 {
3     // 61 references
4     public class Book
5     {
6         // 4 references
7         private int _bookId;
8         // 4 references
9         private string _title;
10        // 4 references
11        private string _author;
12
13        // 16 references
14        public Book(int bookId, string title, string author)
15        {
16            _bookId = bookId;
17            _title = title;
18            _author = author;
19        }
20
21        // 0 references
22        public int BookId
23        {
24            get { return _bookId; }
25            set { _bookId = value; }
26        }
27
28        // 7 references
29        public string Title
30        {
31            get { return _title; }
32            set { _title = value; }
33        }
34
35        // 2 references
36        public string Author
37        {
38            get { return _author; }
39        }
40    }
41 }
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\MODULE 2\LibraryManagementSystem> dotnet build
Restore complete (0.3s)
LibraryManagementSystem net8.0 succeeded (0.1s) -> bin\Debug\net8.0\LibraryManagementSystem.dll

Build succeeded in 0.9s
PS D:\Cognizant Hands-On\MODULE 2\LibraryManagementSystem> dotnet run
=== Library Management System ===

1. SEARCH ALGORITHMS EXPLANATION
=====

What is Search Algorithm?
- A search algorithm finds a specific item in a data structure
- Returns the item if found, or indicates not found
- Critical for library systems to find books quickly

Different Search Algorithms:

1. LINEAR SEARCH
- Checks each element one by one from start to end
- Stops when item is found or end is reached
- Time Complexity: O(n)
- Works on unsorted and sorted data
- Simple to implement
- Example: [5,3,8,1,9] search for 8
  check 5 (no), check 3 (no), check 8 (yes!)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\MODULE 2\LibraryManagementSystem> dotnet run
Comparison Table:



| Algorithm     | Time(Avg) | Sorted Req | Simple    |
|---------------|-----------|------------|-----------|
| Linear Search | O(n)      | No         | Very Easy |
| Binary Search | O(log n)  | Yes        | Moderate  |



How Linear Search Works:
1. Start at first element (index 0)
2. Compare current element with target
3. If match, return the element
4. If no match, move to next element
5. Repeat until found or end reached
6. If not found, return null

How Binary Search Works:
1. Ensure array is sorted
2. Set left = 0, right = n-1
3. Calculate mid = (left + right) / 2
4. Compare middle element with target
5. If match, return the element
6. If target > mid, search right half (left = mid + 1)
7. If target < mid, search left half (right = mid - 1)
8. Repeat until found or left > right

Visual Example:

Linear Search: [5,3,8,1,9] search for 8
[5,3,8,1,9] Compare 5 != 8
[5,3,8,1,9] Compare 3 != 8
[5,3,8,1,9] Compare 8 == 8 ✓ FOUND!
Comparisons: 3

Binary Search: [1,3,5,8,9] search for 8
[1,3,5,8,9] left=0, right=4, mid=2, value=5
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\MODULE 2\LibraryManagementSystem> dotnet run

Binary Search: [1,3,5,8,9] search for 8
[1,3,5,8,9] left=0, right=4, mid=2, value=5
5 < 8, search right half
[1,3,5,8,9] left=3, right=4, mid=3, value=8
8 == 8 ✓ FOUND!
Comparisons: 2

2. LINEAR SEARCH TESTS
=====

=== Book List ===
Total Books: 8
-----
1. ID: 1, Title: The Great Gatsby, Author: F. Scott Fitzgerald
2. ID: 2, Title: To Kill a Mockingbird, Author: Harper Lee
3. ID: 3, Title: 1984, Author: George Orwell
4. ID: 4, Title: Pride and Prejudice, Author: Jane Austen
5. ID: 5, Title: The Catcher in the Rye, Author: J.D. Salinger
6. ID: 6, Title: Animal Farm, Author: George Orwell
7. ID: 7, Title: Brave New World, Author: Aldous Huxley
8. ID: 8, Title: The Hobbit, Author: J.R.R. Tolkien
-----

Test 1: Search by Title (Exact Match)
Found: ID: 3, Title: 1984, Author: George Orwell

Test 2: Search by Title (Partial Match)
Found 1 book(s):
    ID: 6, Title: Animal Farm, Author: George Orwell

Test 3: Search by Author
Found: ID: 3, Title: 1984, Author: George Orwell
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\MODULE 2\LibraryManagementSystem> dotnet run

3. BINARY SEARCH TESTS
=====

=== Book List ===
Total Books: 8
-----
1. ID: 1, Title: The Great Gatsby, Author: F. Scott Fitzgerald
2. ID: 2, Title: To Kill a Mockingbird, Author: Harper Lee
3. ID: 3, Title: 1984, Author: George Orwell
4. ID: 4, Title: Pride and Prejudice, Author: Jane Austen
5. ID: 5, Title: The Catcher in the Rye, Author: J.D. Salinger
6. ID: 6, Title: Animal Farm, Author: George Orwell
7. ID: 7, Title: Brave New World, Author: Aldous Huxley
8. ID: 8, Title: The Hobbit, Author: J.R.R. Tolkien
-----

=== Sorted Books (by Title) ===
Total Books: 8
-----
1. ID: 3, Title: 1984, Author: George Orwell
2. ID: 6, Title: Animal Farm, Author: George Orwell
3. ID: 7, Title: Brave New World, Author: Aldous Huxley
4. ID: 4, Title: Pride and Prejudice, Author: Jane Austen
5. ID: 5, Title: The Catcher in the Rye, Author: J.D. Salinger
6. ID: 1, Title: The Great Gatsby, Author: F. Scott Fitzgerald
7. ID: 8, Title: The Hobbit, Author: J.R.R. Tolkien
8. ID: 2, Title: To Kill a Mockingbird, Author: Harper Lee
-----

Test 1: Binary Search (Middle Element)
Found: ID: 4, Title: Pride and Prejudice, Author: Jane Austen

Test 2: Binary Search (First Element)
Found: ID: 3, Title: 1984, Author: George Orwell
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\MODULE 2\LibraryManagementSystem> dotnet run
- Can search by partial title/author

For Medium Library (500-10,000 books):
USE: Binary Search (if sorted) or Linear (if not)
- Sort books by title once
- Use binary search for fast lookups
- Or use linear for partial matches

For Large Library (> 10,000 books):
USE: Binary Search + Sorted Index
- Maintain sorted book list
- Binary search for  $O(\log n)$  lookups
- Or use Dictionary for  $O(1)$  lookups

BEST PRACTICE FOR PRODUCTION:
Use Dictionary (HashMap):
Dictionary<int, Book> booksById = new Dictionary<int, Book>();
Dictionary<string, Book> booksByTitle = new Dictionary<string, Book>();
- Add: booksByTitle.Add(title, book) -  $O(1)$ 
- Search: booksByTitle[title] -  $O(1)$ 
- Best for all sizes!

COMPLETE COMPARISON TABLE:
-----
```

Algorithm	Time(Avg)	Time(Worst)	Sorted	Use Case
Linear	$O(n)$	$O(n)$	No	Small
Binary	$O(\log n)$	$O(\log n)$	Yes	Large
Dictionary	$O(1)$	$O(1)$	No	BEST

```
=== Library Management Complete ===
PS D:\Cognizant Hands-On\MODULE 2\LibraryManagementSystem>
```